

ASSERTIONS, ASSUMPTIONS AND COVERS AHB-LITE PROTOCOL								
Property Name	Reference	Description	⊙	Status	⊙	Obligation	File	FAILURE REASON
ISL_ZERO_WAIT	3.2 Table 3-1	Slaves must always provide a zero wait state OKAY response to DLE transfers. Transfer must be ignored by the slave.	Written	Passing	Slave	Always	ahb_checker.py	
HRESP_ALWAYS_OKAY	5.1 Table 5-1	HRESP=LOW indicates OKAY. This DUT hardwires HRESP=HRESP_OKAY unconditionally – must never be HIGH.	Written	Passing	Slave	Always	ahb_checker.py	
HREADYOUT_DURING_RESET	7.1.2	During reset all slaves must ensure HREADYOUT is HIGH. Active-low reset: when HRESET-to-0, HREADYOUT must be 1.	Written	Vacuous	Slave	Always	ahb_checker.py	enters is HRESET0. If your reset handling in TGL means HRESET is never seen as low after elaboration, this cover shows zero hits. This is fine
HRESP_ERROR_TWICE_CYCLE	5.1.3	ERROR response requires exactly 2 cycles: Cycle 1 HRESP=1 and HREADY=0, Cycle 2 HRESP=1 and HREADY=1.	Written	Vacuous	Slave	Always	ahb_checker.py	Since HRESP is hardwired to OKAY this will never fire. This is expected and correct
HRESP_OKAY_DURING_WAIT	5.1.2	When slave waits wait states prior to completing the response, it must drive HRESP = OKAY throughout.	Written	Passing	Slave	Always	ahb_checker.py	
HRESP_STABLE_8CYCLES	Section 5.1.3	To start the ERROR response, the slave drives HRESP to indicate ERROR while driving HREADY LOW to extend the transfer for one extra cycle. In the next cycle HREADY is shown HIGH and the transfer and HRESP remains down HIGH to indicate ERROR.	Written	Vacuous	Slave	Always	ahb_checker.py	Since HRESP is hard coded to be OKAY this is a vacuous pass
NO_NSEQ_AFTER_DLE	DUT Specific 3.6	This DUT inserts exactly 1 wait state on every NONSEQ read. HREADYOUT must go LOW for exactly 1 cycle then return HIGH.	Written	Vacuous	Slave	Always	ahb_checker.py	The DUT intent is to have a min 1 wait state (the design has no min but 16 max limit - recommended) But DUT has a precedence issue causing waits without keeping in mind ahb_read. So it is not a spec BUG rather a design intention BUG
WRITE_NO_WAIT_STATE	DUT Specific 3.1	This DUT never inserts wait states on write transfers. HREADYOUT must remain HIGH on all write transfers.	Written	Passing	Slave	Always	ahb_checker.py	
SEQ_READ_NO_WAIT_STATE	DUT Specific 4.3	Single slave system – HREADY input is directly driven by HREADYOUT output. These must always be equal.	Written	Passing	Slave	Always	ahb_checker.py	
HDATA_VALID_ON_COMPLETE	Section 6.1.2	A slave only has to provide valid data when a transfer completes with an OKAY response. ERROR responses do not require valid read data.	Written	Passing	Slave	Always	ahb_checker.py	
DATA_VALIDITY_PROP	Document Part 2 - Functional Properties	This is the bug. The memory user would be for both reading and writing. But for a read, the correct address should be valid (the same) This is a Confirmed DUT Bug	Written	Failing	Slave	Always	ahb_checker.py	This is the bug. The memory user would be for both reading and writing. But for a read, the correct address should be valid (the current HADDR0, or valid the previous cycle's HADDR0). When a read happens at cycle 70 address H0FC, valid has already moved to H07C (updated at end of cycle 7). So it must return the contents of H07C, not H0FC. The read comes back from the wrong address ending (the H07C) proving that the Slave was looking at the wrong memory location during the completion of the handshake
MEM_STABILITY_PROP	Document Part 2 - Functional Properties	No memory location changes without a valid write transaction	Written	Failing	Slave	Always	ahb_checker.py	The MEM_STABILITY assertion failed because the data returned (H0FC) did not match the actual value stored at the requested address (H07C), proving that the Slave was looking at the wrong memory location during the completion of the handshake
BYTE_WRITE_ISOLATION_PROP	Document Part 2 - Functional Properties	Byte write to address A does not modify bytes A+1, A+2, A+3	Not properly checked	Slave	Always	Always	ahb_checker.py	Could not verify whether an actual bug or an issue with the assertion
RESET_STAYS_HIGH	Not specified	This is added because HRESETn was asserted initially for testing and this then deasserts it throughout the assumptions file	Written	Passing	Master	Always	ahb_assertions.py	
ASSUME_HSEL_ALWAYS_ON	DUT limitation	Since we have a single slave, the MUX only has to select 1 slave hence HSEL is always 1. This is added to keep assumptions and cover	Written	Passing	Master	Always	ahb_assertions.py	
SEQ_FOLLOWING_NONSEQ_OR_SEQ	3.2 Table 3-1	SEQ transfer can only follow NONSEQ or SEQ. SEQ cannot appear after DLE or a burst start without preceeding NONSEQ	Written	Passing	Master	Always	ahb_assertions.py	
NO_SEQ_AFTER_DLE	3.2 Table 3-1	SEQ cannot appear as the very first transfer. Must always be preceded by a NONSEQ that started the burst	Written	Passing	Master	Always	ahb_assertions.py	
NO_BUSY_AFTER_SINGLE	3.5.1	Master is not permitted to perform a BUSY transfer immediately after a SINGLE burst. Must go DLE or NONSEQ	Written	Passing	Master	Always	ahb_assertions.py	
FIXED_BURST_NO_BUSY_LAST	//	Fixed length bursts (INC8/8/16, WRAP4/8/16) must not end with BUSY. Last burst must be SEQ.	Written	Passing	Master	Always	ahb_assertions.py	
FIXED_BURST_COUNT_INC84	//	INC84 must complete exactly 4 beats: 1 NONSEQ followed by exactly 3 SEQ beats.	Written	Passing	Master	Always	ahb_assertions.py	
FIXED_BURST_COUNT_WRAP4	//	WRAP4 must complete exactly 4 beats: 1 NONSEQ followed by exactly 3 SEQ beats.	Written	Passing	Master	Always	ahb_assertions.py	
FIXED_BURST_COUNT_INC88	//	INC88 must complete exactly 8 beats: 1 NONSEQ followed by exactly 7 SEQ beats.	Written	Passing	Master	Always	ahb_assertions.py	
FIXED_BURST_COUNT_WRAP8	//	WRAP8 must complete exactly 8 beats: 1 NONSEQ followed by exactly 7 SEQ beats.	Written	Passing	Master	Always	ahb_assertions.py	
FIXED_BURST_COUNT_INC164	//	INC164 must complete exactly 16 beats: 1 NONSEQ followed by exactly 15 SEQ beats.	Written	Passing	Master	Always	ahb_assertions.py	
FIXED_BURST_COUNT_WRAP16	//	WRAP16 must complete exactly 16 beats: 1 NONSEQ followed by exactly 15 SEQ beats.	Written	Passing	Master	Always	ahb_assertions.py	
HTRANS_STABLE_NONSEQ_SEQ	3.6	If HTRANS is NONSEQ or SEQ, it must remain stable if HREADY is LOW.	Written	Passing	Master	Always	ahb_assertions.py	
HTRANS_WAIT_DLE_TO_NONSEQ	//	During a waited transfer, DLE can only change to NONSEQ	Written	Passing	Master	Always	ahb_assertions.py	
HTRANS_WAIT_BUSY_TO_SEQ	//	During a waited fixed length burst, BUSY can only change to SEQ	Written	Passing	Master	Always	ahb_assertions.py	
HTRANS_WAIT_BUSY_ANY	//	During a waited INC8 BURST, BUSY can change to any type	Written	Passing	Master	Always	ahb_assertions.py	
HADDR_STABLE_DURING_WAIT	3.6.2	When slave requests wait states (HREADY=0), master must not change HADDR until HREADY returns HIGH	Written	Passing	Master	Always	ahb_assertions.py	
HWRITE_STABLE_DURING_WAIT	3.6	When slave requests wait states (HREADY=0), master must not change HWRITE direction signal.	Written	Passing	Master	Always	ahb_assertions.py	
HWRITE_STABLE_DURING_WAIT	//	When slave requests wait states (HREADY=0), master must not change HSIZ2 transfer size.	Written	Passing	Master	Always	ahb_assertions.py	
HBURST_STABLE_DURING_WAIT	//	When slave requests wait states (HREADY=0), master must not change HBURST burst type	Written	Passing	Master	Always	ahb_assertions.py	
HADDR_ALIGNED_HWORD	3.5 Table 3-3	Halfword transfers (HSIZ2=HWORD) must have HADDR[0]=0. Address must be aligned to 2-byte boundary	Written	Passing	Master	Always	ahb_assertions.py	
HADDR_ALIGNED_WORD	3.5 Table 3-3	Word transfers (HSIZ2=WORD) must have HADDR[0]=0-3. Address must be aligned to 4-byte boundary	Written	Passing	Master	Always	ahb_assertions.py	
HSIZE_LEGAL_FOR_BUS	3.4	Transfer size must not exceed bus width. With 32-bit data bus, HSIZ2 must be BYTE, HWORD or WORD only.	Written	Passing	Master	Always	ahb_assertions.py	
HWRITE_CONSTANT_BURST	2.2	HWRITE must remain constant throughout a burst transfer. Direction cannot change between beats.	Written	Passing	Master	Always	ahb_assertions.py	
HBURST_CONSTANT_BURST	2.4	HBURST must remain constant throughout a burst transfer. Burst type cannot change mid-burst.	Written	Passing	Master	Always	ahb_assertions.py	
HSIZE_CONSTANT_BURST	//	HSIZ2 must remain constant throughout a burst transfer. Transfer size cannot change mid-burst.	Written	Passing	Master	Always	ahb_assertions.py	
INC8_NO_HIS_BOUNDARY	4.1	Master is not permitted to start an incrementing burst that crosses a 128 (1024 bytes) address boundary.	Written	Passing	Master	Always	ahb_assertions.py	
HADDR_INCR_INCREMENT	3.2 Table 3-1	For INC84, each SEQ address must equal previous address plus transfer size in bytes (1 ↔ HSIZ2)	Written	Passing	Master	Always	ahb_assertions.py	
HADDR_INCR_INCREMENT	//	For INC84, each SEQ address must equal previous address plus transfer size in bytes (1 ↔ HSIZ2)	Written	Passing	Master	Always	ahb_assertions.py	
HADDR_INCR_INCREMENT	//	For INC16, each SEQ address must equal previous address plus transfer size in bytes (1 ↔ HSIZ2)	Written	Passing	Master	Always	ahb_assertions.py	
HADDR_WRAP4_BOUNDARY	3.5	WRAP4 addresses must stay within naturally aligned boundary. 4 x transfer_size bytes. Address wraps at boundary.	Written	Passing	Master	Always	ahb_assertions.py	
HADDR_WRAP8_BOUNDARY	//	WRAP8 addresses must stay within naturally aligned boundary. 8 x transfer_size bytes. Address wraps at boundary.	Written	Passing	Master	Always	ahb_assertions.py	
HADDR_WRAP16_BOUNDARY	//	WRAP16 addresses must stay within naturally aligned boundary. 16 x transfer_size bytes. Address wraps at boundary.	Written	Passing	Master	Always	ahb_assertions.py	
HSEL_STABLE_DURING_BURST	4.1	HSEL is a combinatorial decode of address bus. Must remain stable throughout entire burst transfer.	Written	Passing	Master	Always	ahb_assertions.py	
HTRANS_DLE_DURING_RESET	7.1.2	During reset all masters must ensure HTRANS=0 indicates DLE. No transfers during active reset.	Written	Separately	Master	Always	ahb_assertions.py	
HPROT_STABLE_IN_BURST	3.7	HPROT has same timing as address signals and must remain constant throughout a burst	Written	Passing	Master	Always	ahb_assertions.py	
HPROT_UNKNOWN_ON_TRANSFER	//	HPROT must not be unknown (XZ) during active NONSEQ or SEQ transfers	Written	Passing	Master	Always	ahb_assertions.py	
BURST_TERMINATED_IN_BURST	3.5.1	Burst termination after a BUSY transfer	Written	Passing	Master	Always	ahb_assertions.py	
HTRANS_NONSEQ_DURING_WAIT	3.6.1	When the HTRANS transfer type changes to NONSEQ the master must keep HTRANS constant, until HREADY is HIGH	Written	Passing	Master	Always	ahb_assertions.py	
HDATA_STABLE_DURING_WAIT	5.1.1	If the transfer is extended then the master must hold the data valid until the transfer completes	Written	Passing	Master	Always	ahb_assertions.py	
SLAVE_REQUESTING_WAIT	3.1	For write operations the master holds the data stable throughout the extended cycles	Written	Passing	Master	Always	ahb_assertions.py	
NORMAL_TRANSFER_START	3.1	Basic Transfer with no wait state check	Written	Passing	Master	Always	ahb_assertions.py	
NORMAL_TRANSFER_BUS_WRITE	3.1	Basic Transfer with no wait state check	Written	Passing	Master	Always	ahb_assertions.py	
SEQ_STABLE_DURING_FIXED_BURST_WAIT	3.5	Once HTRANS changes to SEQ during a fixed burst wait it must stay SEQ	Written	Passing	Master	Always	ahb_assertions.py	
FIXED_BURST_NO_BUSY_LAST	3.5.1	The protocol does not permit a master to end a burst with a BUSY transfer for fixed length bursts of type: incrementing INC84, INC88, and INC16 or wrapping WRAP4, WRAP8, and WRAP16.	Written	Passing	Master	Always	ahb_assertions.py	
NORMAL_TRANSFER_BUS_COMPLETE	3.5.9	Burst termination after a BUSY transfer	Written	Passing	Master	Always	ahb_assertions.py	
COV_HREADY_LOW_3_CYCLES	Document Part 3.2	HREADY=0 held for 3 consecutive cycles	Written	Unreachable	Slave	Always	ahb_cover.py	The DUT's HREADYOUT logic structurally can only hold low for exactly 1 cycle. There is no code path that extends it further.
COV_WRAP4_WRAP_OCCURS	Document Part 3.2	WRAP4 burst where the wrap actually happens	Written	Reachable	Master	Always	ahb_cover.py	
COV_WRITE_THEN_READ_SAME_ADDR	Document Part 3.2	Write immediately followed by read to the same address	Written	Reachable	Master	Always	ahb_cover.py	
COV_HRESP_ERROR	Document Part 3.2	HRESP=ERROR generated by the slave	Written	Unreachable	Slave	Always	ahb_cover.py	he DUT hardwires HRESP = HRESP_OKAY unconditionally. ERROR is structurally impossible.
COV_BACK_TO_BACK_NONSEQ	Document Part 3.2	Back-to-back NONSEQ with no DLE	Written	Reachable	Master	Always	ahb_cover.py	
COV_SEQ_READ_COMPLETES	Self Thought	confirms the no-read state path for SEQ reads is reachable	Written	Reachable	Slave	Always	ahb_cover.py	
COV_CONTENTION_PATH	Self Thought	Back-to-back write then read to same address. This specifically exercises the contention logic path (see ahb_read) - contention registers in the DUT. Different from COV which only checks a consecutive NONSEQ, this confirms the contention resolution path is reachable.	Written	Reachable	Slave	Always	ahb_cover.py	
COV_WRITE_WRITE_OCCURS	Self Thought	confirms HSIZ2_BYTE transfers are reachable	Written	Reachable	Master	Always	ahb_cover.py	
COV_HWORD_WRITE_OCCURS	Self Thought	confirms HSIZ2_HWORD is reachable	Written	Reachable	Master	Always	ahb_cover.py	
COV_WORD_WRITE_OCCURS	Self Thought	confirms HSIZ2_WORD is reachable	Written	Reachable	Master	Always	ahb_cover.py	
COV_INCR_BURST_3_BEATS	Self Thought	INC8 burst of at least 3 beats completes. Exercises the underdefined length burst path through the DUT	Written	Reachable	Master	Always	ahb_cover.py	
COV_WADDR_ADDR_BUS_SCENARIO	Self Thought	What then read to same address with increasing write to different address – exercises the exact scenario where the valid/read bus manifests. DATA_VALIDITY and MEM_STABILITY both caught failures in this pattern.	Written	Reachable	Slave	Always	ahb_cover.py	
COV_BYTE_LANES_READ_BACK	Self Thought	Byte write to each specific lane followed by word read	Written	Reachable	Slave	Always	ahb_cover.py	
COV_BYTE_LANES_READ_BACK	Self Thought	Byte write to each specific lane followed by word read	Written	Reachable	Slave	Always	ahb_cover.py	
COV_BYTE_LANES_READ_BACK	Self Thought	Byte write to each specific lane followed by word read	Written	Reachable	Slave	Always	ahb_cover.py	
COV_BYTE_LANES_READ_BACK	Self Thought	Byte write to each specific lane followed by word read	Written	Reachable	Slave	Always	ahb_cover.py	
ASSUMPTIONS								
ASSERTIONS								
COVERS								